Introduction to data

Guibril Ramde

Some define statistics as the field that focuses on turning information into knowledge. The first step in that process is to summarize and describe the raw information – the data. In this lab we explore flights, specifically a random sample of domestic flights that departed from the three major New York City airports in 2013. We will generate simple graphical and numerical summaries of data on these flights and explore delay times. Since this is a large data set, along the way you'll also learn the indispensable skills of data processing and subsetting.

Getting started

Load packages

In this lab, we will explore and visualize the data using the **tidyverse** suite of packages. The data can be found in the companion package for OpenIntro labs, **openintro**.

Let's load the packages.

```
library(tidyverse)
library(openintro)
```

The data

The Bureau of Transportation Statistics (BTS) is a statistical agency that is a part of the Research and Innovative Technology Administration (RITA). As its name implies, BTS collects and makes transportation data available, such as the flights data we will be working with in this lab.

First, we'll view the `nycflights` data frame. Type the following in your console to load the data:

```
data(nycflights)
```

The data set `nycflights` that shows up in your workspace is a *data matrix*, with each row representing an *observation* and each column representing a *variable*. R calls this data format a **data frame**, which is a term that will be used throughout the labs. For this data set, each *observation* is a single flight.

To view the names of the variables, type the command

```
names(nycflights)
```

```
##  [1] "year"      "month"     "day"       "dep_time"  "dep_delay" "arr_time"
##  [7] "arr_delay" "carrier"   "tailnum"   "flight"    "origin"    "dest"
## [13] "air_time"  "distance"  "hour"      "minute"
```

This returns the names of the variables in this data frame. The **codebook** (description of the variables) can be accessed by pulling up the help file:

```r
?nycflights
```

One of the variables refers to the carrier (i.e. airline) of the flight, which is coded according to the following system.

- `carrier`: Two letter carrier abbreviation.
  - `9E`: Endeavor Air Inc.
  - `AA`: American Airlines Inc.
  - `AS`: Alaska Airlines Inc.
  - `B6`: JetBlue Airways
  - `DL`: Delta Air Lines Inc.
  - `EV`: ExpressJet Airlines Inc.
  - `F9`: Frontier Airlines Inc.
  - `FL`: AirTran Airways Corporation
  - `HA`: Hawaiian Airlines Inc.
  - `MQ`: Envoy Air
  - `OO`: SkyWest Airlines Inc.
  - `UA`: United Air Lines Inc.
  - `US`: US Airways Inc.
  - `VX`: Virgin America
  - `WN`: Southwest Airlines Co.
  - `YV`: Mesa Airlines Inc.

Remember that you can use `glimpse` to take a quick peek at your data to understand its contents better.

```r
glimpse(nycflights)
```

```
## Rows: 32,735
## Columns: 16
## $ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, ~
## $ month     <int> 6, 5, 12, 5, 7, 1, 12, 8, 9, 4, 6, 11, 4, 3, 10, 1, 2, 8, 10~
## $ day       <int> 30, 7, 8, 14, 21, 1, 9, 13, 26, 30, 17, 22, 26, 25, 21, 23, ~
## $ dep_time  <int> 940, 1657, 859, 1841, 1102, 1817, 1259, 1920, 725, 1323, 940~
## $ dep_delay <dbl> 15, -3, -1, -4, -3, -3, 14, 85, -10, 62, 5, 5, -2, 115, -4, ~
## $ arr_time  <int> 1216, 2104, 1238, 2122, 1230, 2008, 1617, 2032, 1027, 1549, ~
## $ arr_delay <dbl> -4, 10, 11, -34, -8, 3, 22, 71, -8, 60, -4, -2, 22, 91, -6, ~
## $ carrier   <chr> "VX", "DL", "DL", "DL", "9E", "AA", "WN", "B6", "AA", "EV", ~
## $ tailnum   <chr> "N626VA", "N3760C", "N712TW", "N914DL", "N823AY", "N3AXAA", ~
## $ flight    <int> 407, 329, 422, 2391, 3652, 353, 1428, 1407, 2279, 4162, 20, ~
## $ origin    <chr> "JFK", "JFK", "JFK", "JFK", "LGA", "LGA", "EWR", "JFK", "LGA~
## $ dest      <chr> "LAX", "SJU", "LAX", "TPA", "ORF", "ORD", "HOU", "IAD", "MIA~
## $ air_time  <dbl> 313, 216, 376, 135, 50, 138, 240, 48, 148, 110, 50, 161, 87,~
## $ distance  <dbl> 2475, 1598, 2475, 1005, 296, 733, 1411, 228, 1096, 820, 264,~
## $ hour      <dbl> 9, 16, 8, 18, 11, 18, 12, 19, 7, 13, 9, 13, 8, 20, 12, 20, 6~
## $ minute    <dbl> 40, 57, 59, 41, 2, 17, 59, 20, 25, 23, 40, 20, 9, 54, 17, 24~
```

The `nycflights` data frame is a massive trove of information. Let's think about some questions we might want to answer with these data:

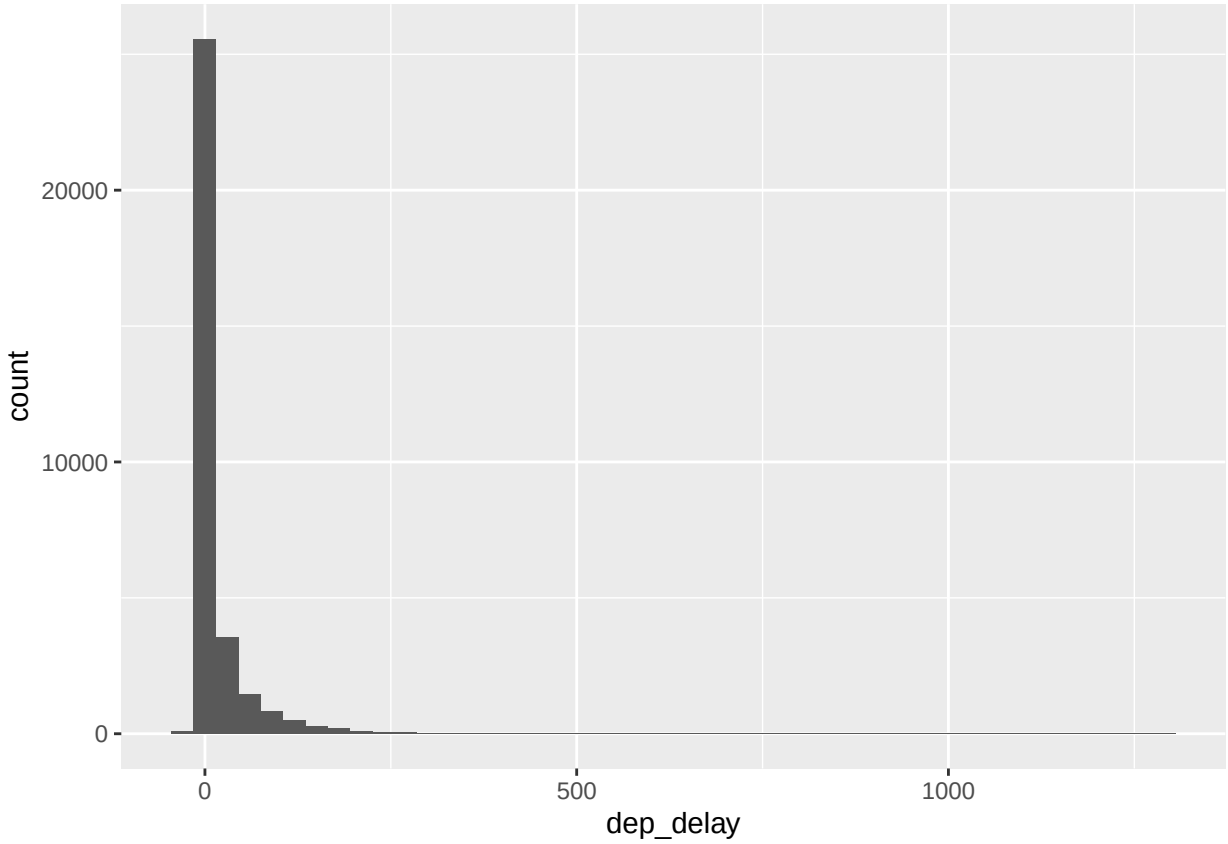
- How delayed were flights that were headed to Los Angeles?
- How do departure delays vary by month?
- Which of the three major NYC airports has the best on time percentage for departing flights?

Analysis

Departure delays

Let's start by examing the distribution of departure delays of all flights with a histogram.

```
ggplot(data = nycflights, aes(x = dep_delay)) +
  geom_histogram(binwidth = 30)
```

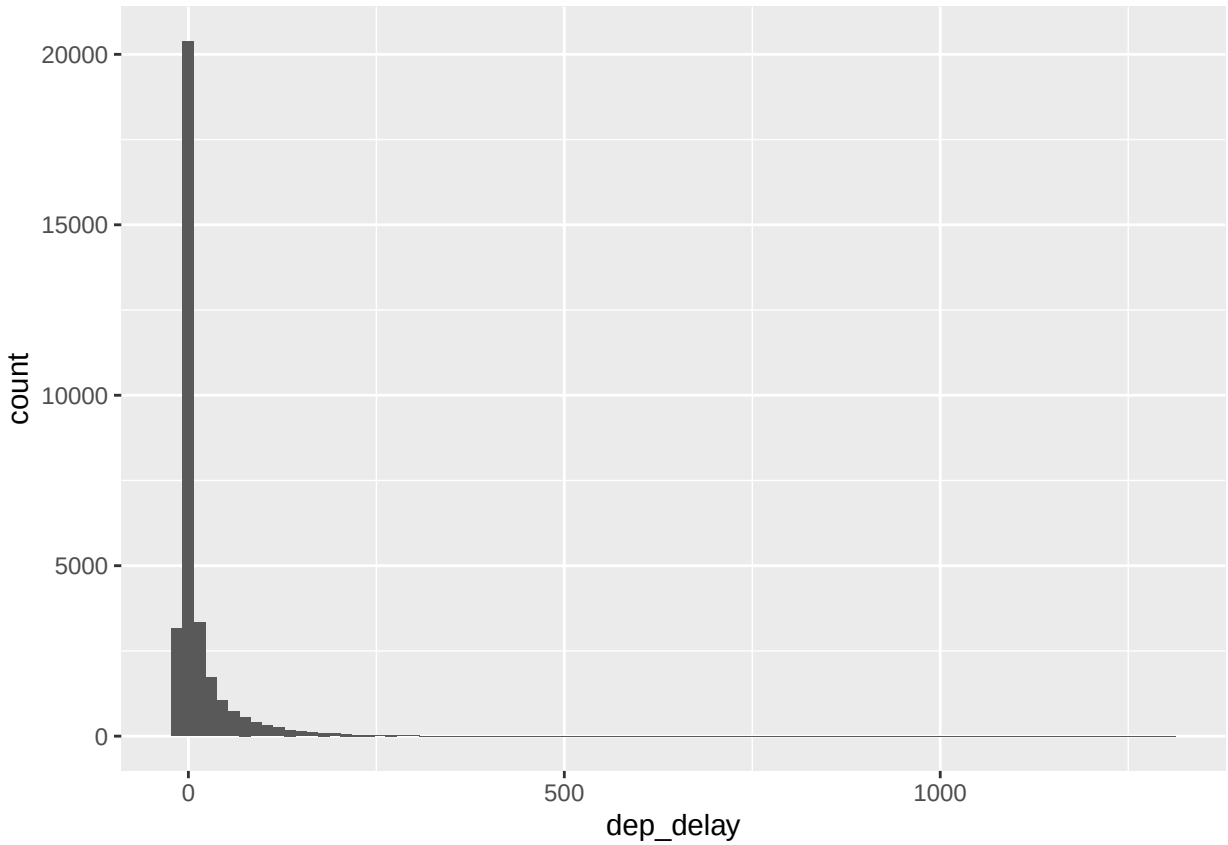



```
#ggsave(filename = "lab1.png")
```

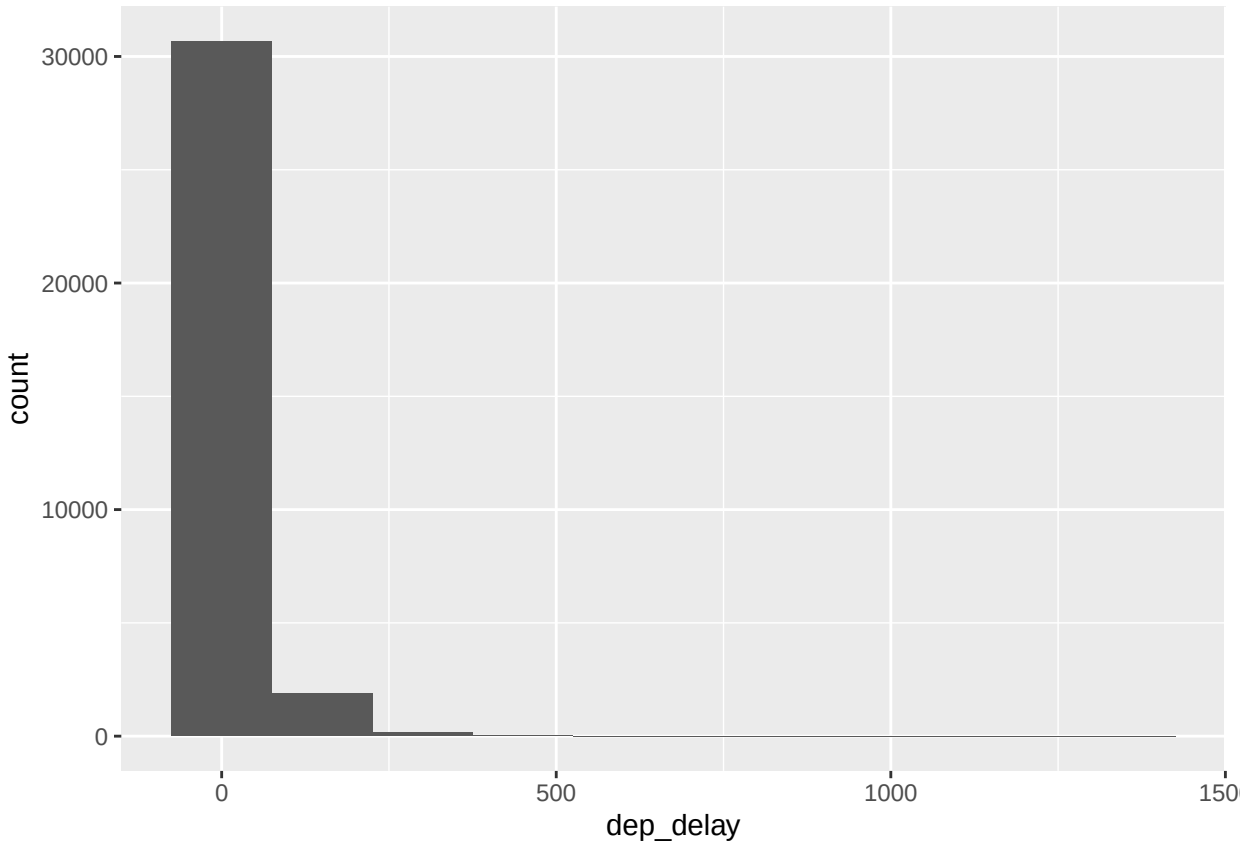
This function says to plot the `dep_delay` variable from the `nycflights` data frame on the x-axis. It also defines a `geom` (short for geometric object), which describes the type of plot you will produce.

Histograms are generally a very good way to see the shape of a single distribution of numerical data, but that shape can change depending on how the data is split between the different bins. You can easily define the binwidth you want to use:

```
ggplot(data = nycflights, aes(x = dep_delay)) +
  geom_histogram(binwidth = 15)
```




```
ggplot(data = nycflights, aes(x = dep_delay)) +
  geom_histogram(binwidth = 150)
```

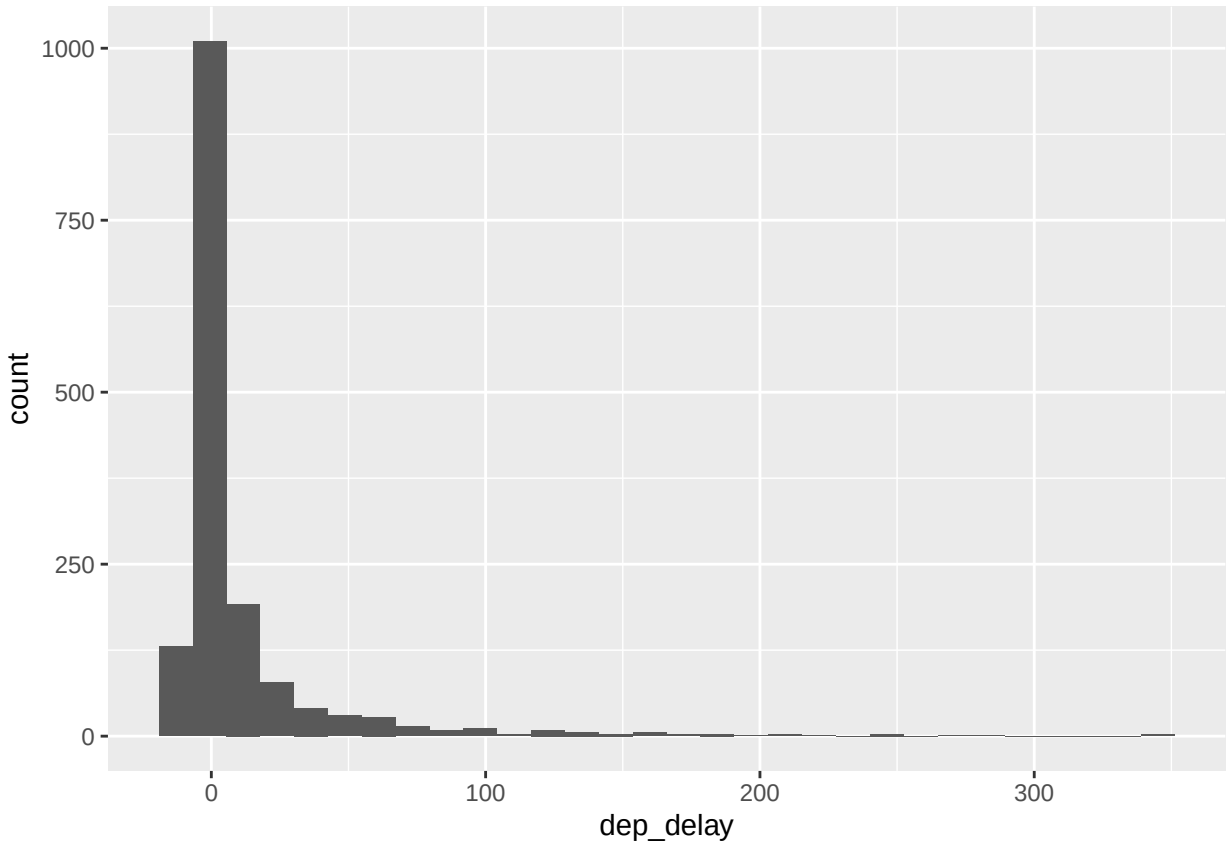


1. Look carefully at these three histograms. How do they compare? Are features revealed in one that are obscured in another?

Insert your answer here Answer these three histograms are very different from each other, with no binwidth define we have more readable bars with the bindth of 15 we get more bars but so tiny also some bars that are visible but we could not see in the first plot with the binwidth of 150 we only see 03 bars which gives less information therefor hard to give correct analysis

If you want to visualize only on delays of flights headed to Los Angeles, you need to first `filter` the data for flights with that destination (`dest == "LAX"`) and then make a histogram of the departure delays of only those flights.

```
lax_flights <- nycflights %>%
  filter(dest == "LAX")
ggplot(data = lax_flights, aes(x = dep_delay)) +
  geom_histogram()
```



Let's decipher these two commands (OK, so it might look like four lines, but the first two physical lines of code are actually part of the same command. It's common to add a break to a new line after `%>%` to help readability).

- Command 1: Take the `nycflights` data frame, `filter` for flights headed to LAX, and save the result as a new data frame called `lax_flights`.
  - `==` means "if it's equal to".
  - `LAX` is in quotation marks since it is a character string.
- Command 2: Basically the same `ggplot` call from earlier for making a histogram, except that it uses the smaller data frame for flights headed to LAX instead of all flights.

Logical operators: Filtering for certain observations (e.g. flights from a particular airport) is often of interest in data frames where we might want to examine observations with certain characteristics separately from the rest of the data. To do so, you can use the `filter` function and a series of **logical operators**. The most commonly used logical operators for data analysis are as follows:

- `==` means "equal to"
- `!=` means "not equal to"
- `>` or `<` means "greater than" or "less than"
- `>=` or `<=` means "greater than or equal to" or "less than or equal to"

You can also obtain numerical summaries for these flights:

```r
lax_flights %>%
  summarise(mean_dd   = mean(dep_delay),
            median_dd = median(dep_delay),
            n         = n())
```

```
## # A tibble: 1 x 3
##   mean_dd median_dd     n
##     <dbl>     <dbl> <int>
## 1    9.78        -1  1583
```

Note that in the `summarise` function you created a list of three different numerical summaries that you were interested in. The names of these elements are user defined, like `mean_dd`, `median_dd`, `n`, and you can customize these names as you like (just don't use spaces in your names). Calculating these summary statistics also requires that you know the function calls. Note that `n()` reports the sample size.

Summary statistics: Some useful function calls for summary statistics for a single numerical variable are as follows:

- `mean`
- `median`
- `sd`
- `var`
- `IQR`
- `min`
- `max`

Note that each of these functions takes a single vector as an argument and returns a single value.

You can also filter based on multiple criteria. Suppose you are interested in flights headed to San Francisco (SFO) in February:

```r
sfo_feb_flights <- nycflights %>%
  filter(dest == "SFO", month == 2)
sfo_feb_flights
```

```
## # A tibble: 68 x 16
##     year month   day dep_time dep_delay arr_time arr_delay carrier tailnum
##    <int> <int> <int>    <int>     <dbl>    <int>     <dbl> <chr>   <chr>
##  1  2013     2    18     1527        57     1903        48 DL      N711ZX
##  2  2013     2     3      613        14     1008        38 UA      N502UA
##  3  2013     2    15      955        -5     1313       -28 DL      N717TW
##  4  2013     2    18     1928        15     2239        -6 UA      N24212
##  5  2013     2    24     1340         2     1644       -21 UA      N76269
##  6  2013     2    25     1415       -10     1737       -13 UA      N532UA
##  7  2013     2     7     1032         1     1352       -10 B6      N627JB
##  8  2013     2    15     1805        20     2122         2 AA      N335AA
##  9  2013     2    13     1056        -4     1412       -13 UA      N532UA
## 10  2013     2     8      656        -4     1039        -6 DL      N710TW
## # i 58 more rows
## # i 7 more variables: flight <int>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>
```

Note that you can separate the conditions using commas if you want flights that are both headed to SFO **and** in February. If you are interested in either flights headed to SFO **or** in February, you can use the `|` instead of the comma.

2. Create a new data frame that includes flights headed to SFO in February, and save this data frame as `sfo_feb_flights`. How many flights meet these criteria?

Insert your answer here 68 flights

```
sfo_feb_flights <- nycflights %>%
  filter(dest == "SFO", month == 2)
sfo_feb_flights
```

```
## # A tibble: 68 x 16
##     year month   day dep_time dep_delay arr_time arr_delay carrier tailnum
##    <int> <int> <int>    <int>     <dbl>    <int>     <dbl> <chr>   <chr>
##  1  2013     2    18     1527        57     1903        48 DL      N711ZX
##  2  2013     2     3      613        14     1008        38 UA      N502UA
##  3  2013     2    15      955        -5     1313       -28 DL      N717TW
##  4  2013     2    18     1928        15     2239        -6 UA      N24212
##  5  2013     2    24     1340         2     1644       -21 UA      N76269
##  6  2013     2    25     1415       -10     1737       -13 UA      N532UA
##  7  2013     2     7     1032         1     1352       -10 B6      N627JB
##  8  2013     2    15     1805        20     2122         2 AA      N335AA
##  9  2013     2    13     1056        -4     1412       -13 UA      N532UA
## 10  2013     2     8      656        -4     1039        -6 DL      N710TW
## # i 58 more rows
## # i 7 more variables: flight <int>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>
```

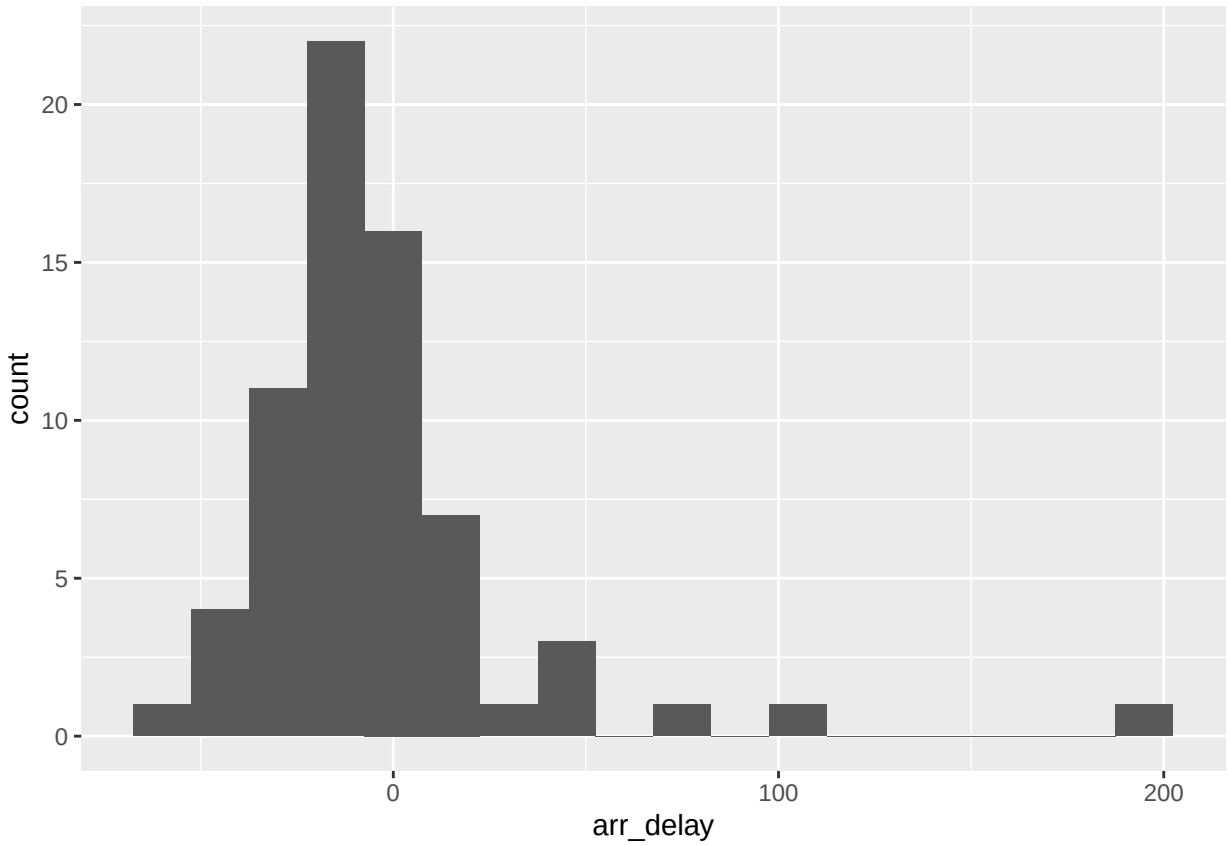
```
nrow(sfo_feb_flights)
```

```
## [1] 68
```

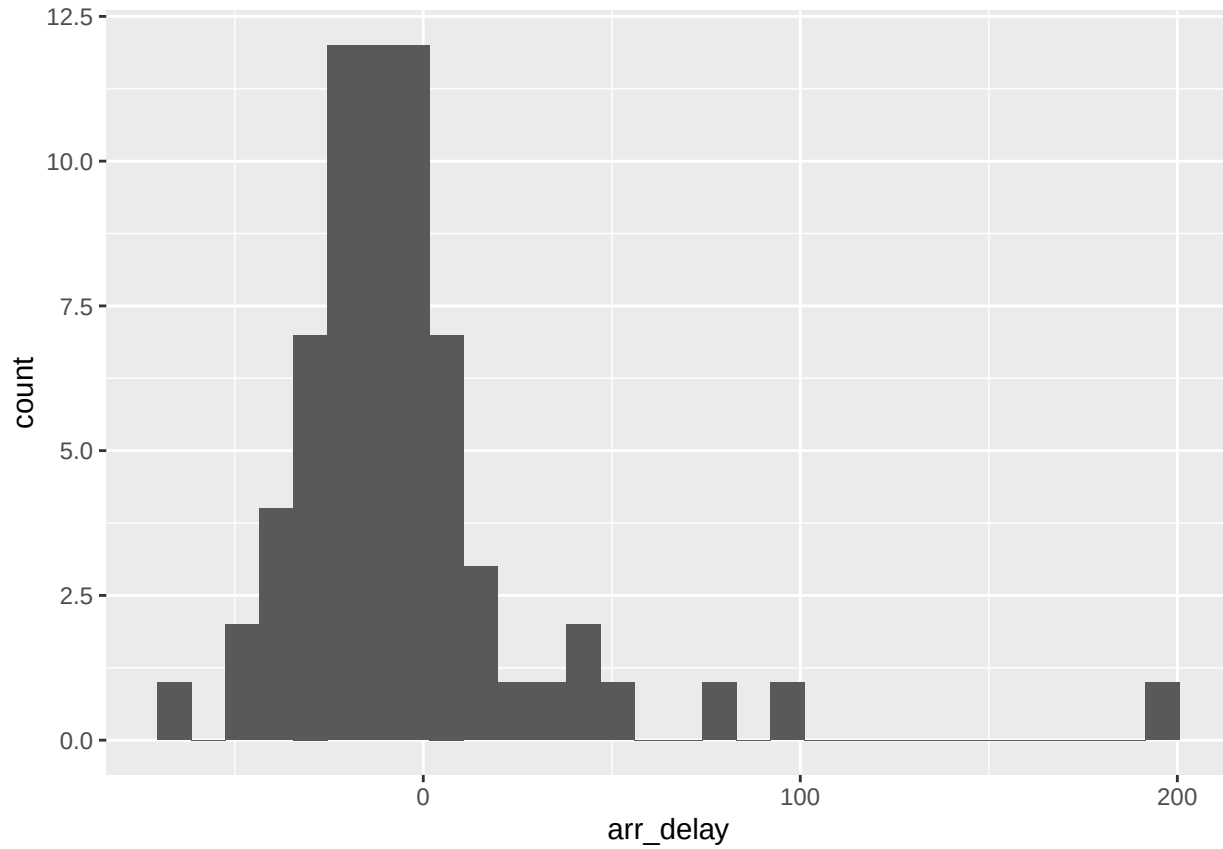
3. Describe the distribution of the **arrival** delays of these flights using a histogram and appropriate summary statistics. **Hint:** The summary statistics you use should depend on the shape of the distribution.

Insert your answer here Answer we can see that the arrival delay on the month of february destination to 'SFO' is not evenly and there are more delay flight.

```
sfo_feb_flights <- nycflights %>%
  filter(dest == "SFO", month == 2)
ggplot(data = sfo_feb_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 15)
```




```
ggplot(data = sfo_feb_flights, aes(x = arr_delay)) +
  geom_histogram()
```




```
sfo_feb_flights <- nycflights %>%
  group_by(origin) %>%
  summarise(median_arr = median(arr_delay, na.rm = TRUE), mean_arr = mean(arr_delay, na.rm = TRUE), iqr_
sfo_feb_flights
```

```
## # A tibble: 3 x 5
##   origin median_arr mean_arr iqr_arr n_flights
##   <chr>       <dbl>    <dbl>   <dbl>     <int>
## 1 EWR            -3     9.33      33     11771
## 2 JFK            -5     5.98      30     10897
## 3 LGA            -5     5.71      29     10067
```

Another useful technique is quickly calculating summary statistics for various groups in your data frame. For example, we can modify the above command using the `group_by` function to get the same summary stats for each origin airport:

```
sfo_feb_flights <- nycflights %>%
  filter(dest == "SFO", month == 2)

sfo_feb_flights %>%
  group_by(origin) %>%
  summarise(median_dd = median(dep_delay), iqr_dd = IQR(dep_delay), n_flights = n())
```

```
## # A tibble: 2 x 4
```

```
##   origin median_dd iqr_dd n_flights
##   <chr>      <dbl>  <dbl>     <int>
## 1 EWR          0.5   5.75         8
## 2 JFK         -2.5  15.2         60
```

Here, we first grouped the data by `origin` and then calculated the summary statistics. **book**

```
get_flights <- nycflights |>
  #count(origin, dest, sort = TRUE)
  #filter(arr_delay >= 120)
  #filter(dest %in% c("IAH", "HOU"))
  #filter(carrier == "UA" & carrier == "AA" | carrier == "DL")
  #filter(month == "7" & month == "8" | month =="9")
  #filter(arr_time > 120, dep_delay <= 0)
  filter(dep_delay >= 60 , dep_delay - arr_delay >= 30)



get_flights
```

```
## # A tibble: 225 x 16
##     year month   day dep_time dep_delay arr_time arr_delay carrier tailnum
##    <int> <int> <int>    <int>     <dbl>    <int>     <dbl> <chr>   <chr>
##  1  2013     8    28     2153       143       13       109 DL      N387DA
##  2  2013     5     9     1344       179     1646       146 VX      N621VA
##  3  2013     4    18       14       149      253       108 AA      N329AA
##  4  2013     7    23     1848       103     2130        70 AA      N3FAAA
##  5  2013    12    26     1224       175     1320       143 EV      N611QX
##  6  2013     5    17     1954       174     2233       142 DL      N325NB
##  7  2013     5    13     1756        61     2020         0 VX      N637VA
##  8  2013     5    28     2216       227       19       185 UA      N654UA
##  9  2013     4    12     1243       188     1355       155 WN      N269WN
## 10  2013     1    24     2213       128       35        97 UA      N405UA
## # i 215 more rows
## # i 7 more variables: flight <int>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>
```

4. Calculate the median and interquartile range for `arr_delay`s of flights in in the `sfo_feb_flights` data frame, grouped by carrier. Which carrier has the most variable arrival delays?

Insert your answer here Answer the carrier that has the most variable delays is 'UA' with 5770 fligths

```
sfo_feb_flights <- nycflights %>%
  group_by(carrier) %>%
  summarise(median_arr = median(arr_delay, na.rm = TRUE), mean_arr = mean(arr_delay, na.rm = TRUE), iqr_
  arrange(desc(iqr_arr))
sfo_feb_flights
```

```
## # A tibble: 16 x 5
##    carrier median_arr mean_arr iqr_arr n_flights
##    <chr>        <dbl>    <dbl>   <dbl>     <int>
##  1 OO            -5     16.7      41.5         3
```

```
## 2 YV            -5     15.1      40          53
## 3 EV             0     16.2      39        5142
## 4 9E            -7      8.04     37        1696
## 5 AS           -19.5  -11.3      33.5        66
## 6 B6            -3      9.73     32        5376
## 7 FL             5     19.6      32         307
## 8 F9             0     12.5      31          69
## 9 VX            -8      2.41     31         497
## 10 AA           -9      1.40     30        3188
## 11 UA           -6      4.36     30        5770
## 12 WN           -3      8.88     30        1261
## 13 MQ           -1     10.1      29        2507
## 14 DL           -8      0.907    27        4751
## 15 US           -6      1.80     23        2015
## 16 HA           -6     28.1      16.8        34
```

Departure delays by month

Which month would you expect to have the highest average delay departing from an NYC airport?

Let's think about how you could answer this question:

- First, calculate monthly averages for departure delays. With the new language you are learning, you could
  - `group_by` months, then
  - `summarise` mean departure delays.
- Then, you could to `arrange` these average delays in `desc`ending order **Answer** Based on the grouped summary statistics arranged in descending order of mean departure delay, July has the highest average departure delay among all months.

```
nycflights %>%
  group_by(month) %>%
  summarise(mean_dd = mean(dep_delay, na.rm = TRUE), median_dd = median(dep_delay, na.rm = TRUE), iqr_d
  arrange(desc(mean_dd))
```

```
## # A tibble: 12 x 5
##    month mean_dd median_dd iqr_delays num_delays
##    <int>   <dbl>     <dbl>      <dbl>      <int>
##  1     7   20.8          0         26       2742
##  2     6   20.4          0         25       2732
##  3    12   17.4          1         25       2716
##  4     4   14.6         -2         16       2781
##  5     3   13.5         -1         17       2869
##  6     5   13.3         -1         19       2821
##  7     8   12.6         -1         15       2880
##  8     2   10.7         -2         15       2286
##  9     1   10.2         -2         12       2610
## 10     9    6.87        -3          8       2681
## 11    11    6.10        -2         10       2733
## 12    10    5.88        -3          9       2884
```

5. Suppose you really dislike departure delays and you want to schedule your travel in a month that minimizes your potential departure delay leaving NYC. One option is to choose the month with the lowest mean departure delay. Another option is to choose the month with the lowest median departure delay. What are the pros and cons of these two choices?

Insert your answer here Answer Choosing the month with the lowest mean departure delay may not necessarily minimize the typical delay experience because the mean is sensitive to extreme values. A few very large delays can increase the mean significantly.

On the other hand, choosing the month with the lowest median departure delay reflects the typical flight more accurately, since the median is resistant to extreme outliers.

Pros of using the mean: Uses all data values, reflects the overall average delay.

Cons of using the mean: Highly influenced by extreme delays (outliers).

Pros of using the median: Resistant to extreme delays, better represents the "typical" departure experience.

Cons of using the median: Does not reflect how severe the largest delays can be.

```
nycflights %>%
  group_by(month)%>%
  summarise(mean_dd = mean(dep_delay), median_dd = median(dep_delay), iqr_delays = IQR(dep_delay), num_
arrange(mean_dd)
```

```
## # A tibble: 12 x 5
##    month mean_dd median_dd iqr_delays num_flight
##    <int>   <dbl>     <dbl>      <dbl>      <int>
##  1    10    5.88        -3          9       2884
##  2    11    6.10        -2         10       2733
##  3     9    6.87        -3          8       2681
##  4     1   10.2         -2         12       2610
##  5     2   10.7         -2         15       2286
##  6     8   12.6         -1         15       2880
##  7     5   13.3         -1         19       2821
##  8     3   13.5         -1         17       2869
##  9     4   14.6         -2         16       2781
## 10    12   17.4          1         25       2716
## 11     6   20.4          0         25       2732
## 12     7   20.8          0         26       2742
```

On time departure rate for NYC airports

Suppose you will be flying out of NYC and want to know which of the three major NYC airports has the best on time departure rate of departing flights. Also supposed that for you, a flight that is delayed for less than 5 minutes is basically "on time." " You consider any flight delayed for 5 minutes of more to be "delayed".

In order to determine which airport has the best on time departure rate, you can

- first classify each flight as "on time" or "delayed",
- then group flights by origin airport,
- then calculate on time departure rates for each origin airport,
- and finally arrange the airports in descending order for on time departure percentage.

Let's start with classifying each flight as "on time" or "delayed" by creating a new variable with the `mutate` function.

```r
nycflights <- nycflights %>%
  mutate(dep_type = ifelse(dep_delay < 5, "on time", "delayed"))
```

The first argument in the `mutate` function is the name of the new variable we want to create, in this case `dep_type`. Then if `dep_delay < 5`, we classify the flight as `"on time"` and `"delayed"` if not, i.e. if the flight is delayed for 5 or more minutes.

Note that we are also overwriting the `nycflights` data frame with the new version of this data frame that includes the new `dep_type` variable.

We can handle all of the remaining steps in one code chunk:

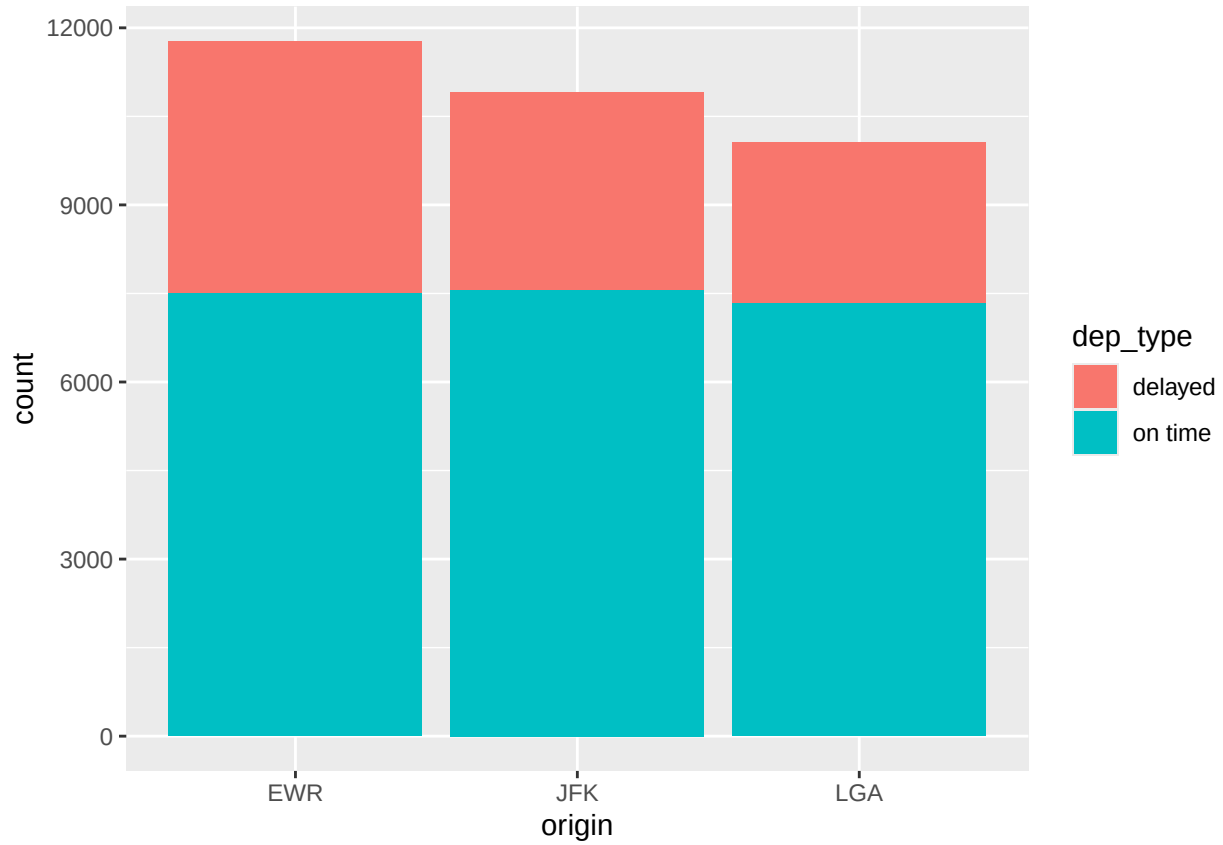
```r
nycflights %>%
  group_by(origin) %>%
  summarise(ot_dep_rate = sum(dep_type == "on time") / n()) %>%
  arrange(desc(ot_dep_rate))
```

```
## # A tibble: 3 x 2
##   origin ot_dep_rate
##   <chr>        <dbl>
## 1 LGA          0.728
## 2 JFK          0.694
## 3 EWR          0.637
```

6. If you were selecting an airport simply based on on time departure percentage, which NYC airport would you choose to fly out of?

You can also visualize the distribution of on on time departure rate across the three airports using a segmented bar plot.

```r
ggplot(data = nycflights, aes(x = origin, fill = dep_type)) +
  geom_bar()
```



Insert your answer here Based on the calculated on-time departure rates, LaGuardia (LGA) has the highest on-time percentage at 72.8%, followed by JFK (69.4%) and EWR (63.7%). Therefore, if selecting an airport solely based on on-time departure percentage, LGA would be the best choice.

Although the bar plot shows that EWR and JFK have higher total counts of on-time flights, those airports also have a larger total number of flights overall. Since we are comparing percentages rather than raw counts, LGA remains the best option. * * *

More Practice

7. Mutate the data frame so that it includes a new variable that contains the average speed, `avg_speed` traveled by the plane for each flight (in mph). **Hint:** Average speed can be calculated as distance divided by number of hours of travel, and note that `air_time` is given in minutes.

Insert your answer here

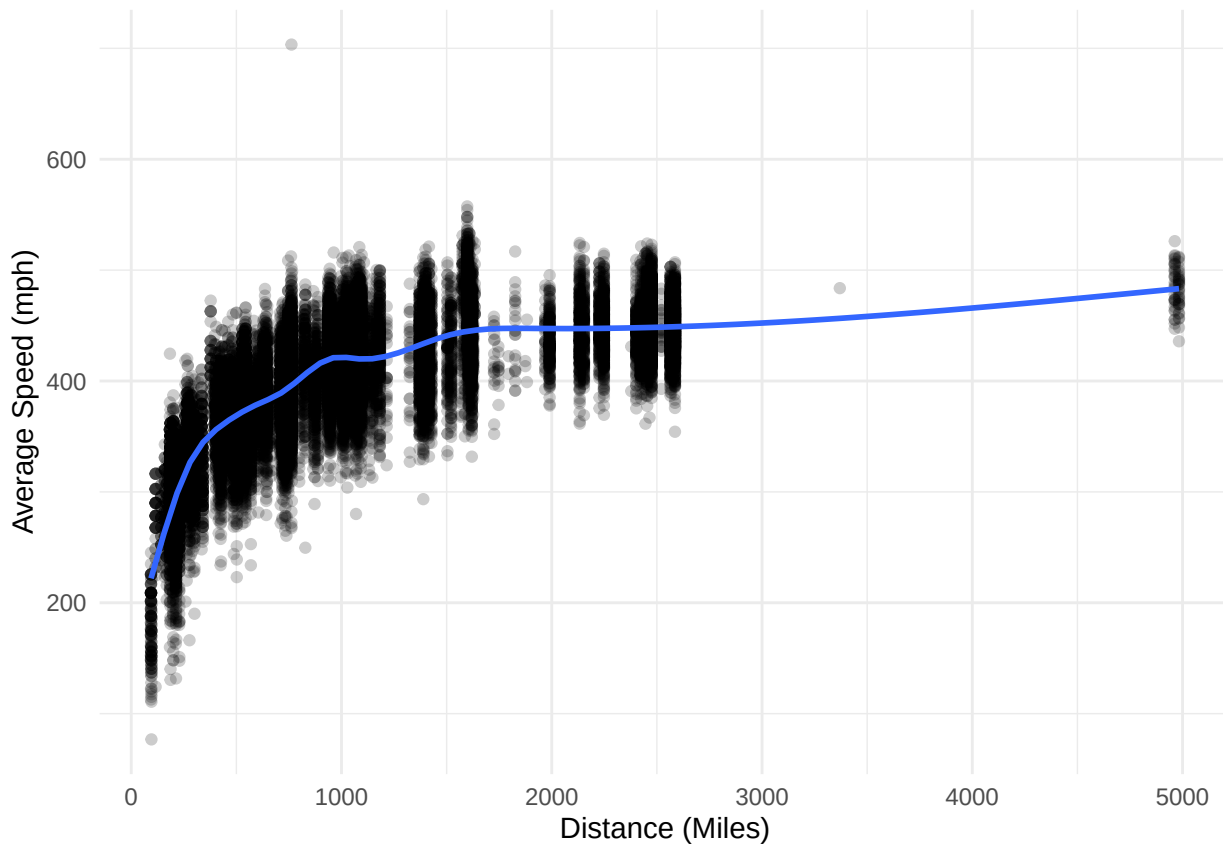
```
nycflights <- nycflights %>%

  mutate(avg_speed = distance /(air_time / 60 ))%>%
arrange(desc(avg_speed))
```

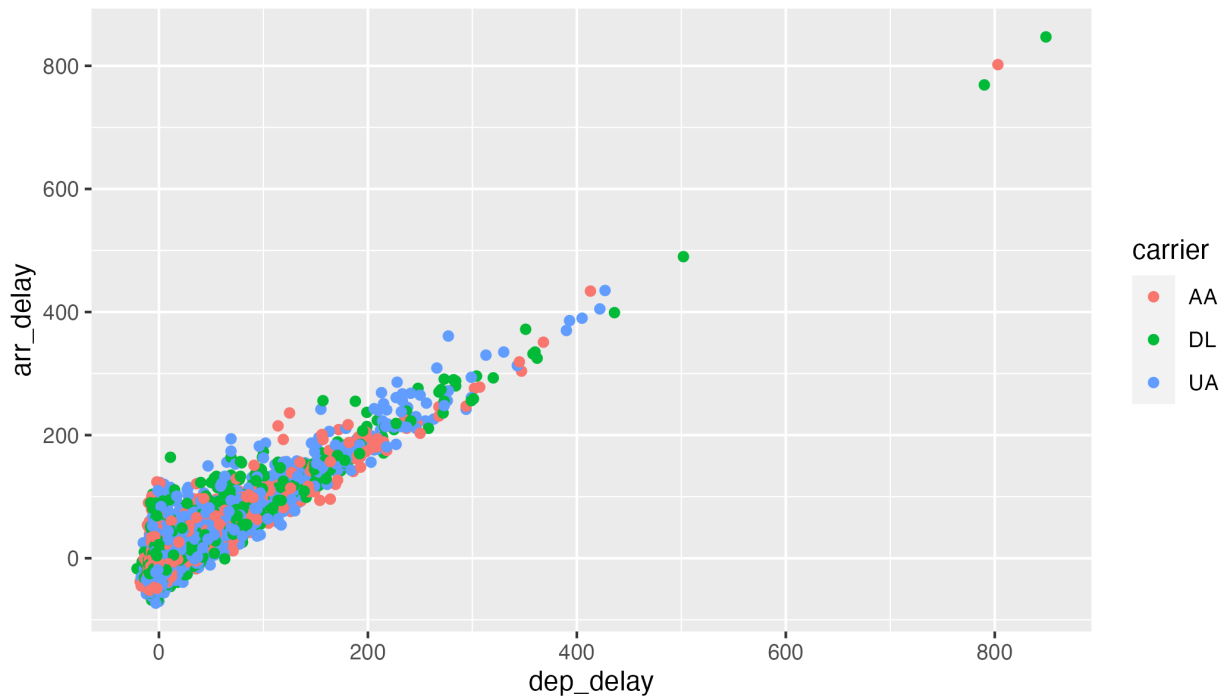
8. Make a scatterplot of `avg_speed` vs. `distance`. Describe the relationship between average speed and distance. **Hint:** Use `geom_point()`.

Insert your answer here

```r
nycflights %>%
  ggplot(aes( x= distance, y = avg_speed)) +
  geom_point(alpha= 0.2) +
  geom_smooth(se = FALSE) +
  labs(x = "Distance (Miles)", y = "Average Speed (mph)" ) +
  theme_minimal()
```

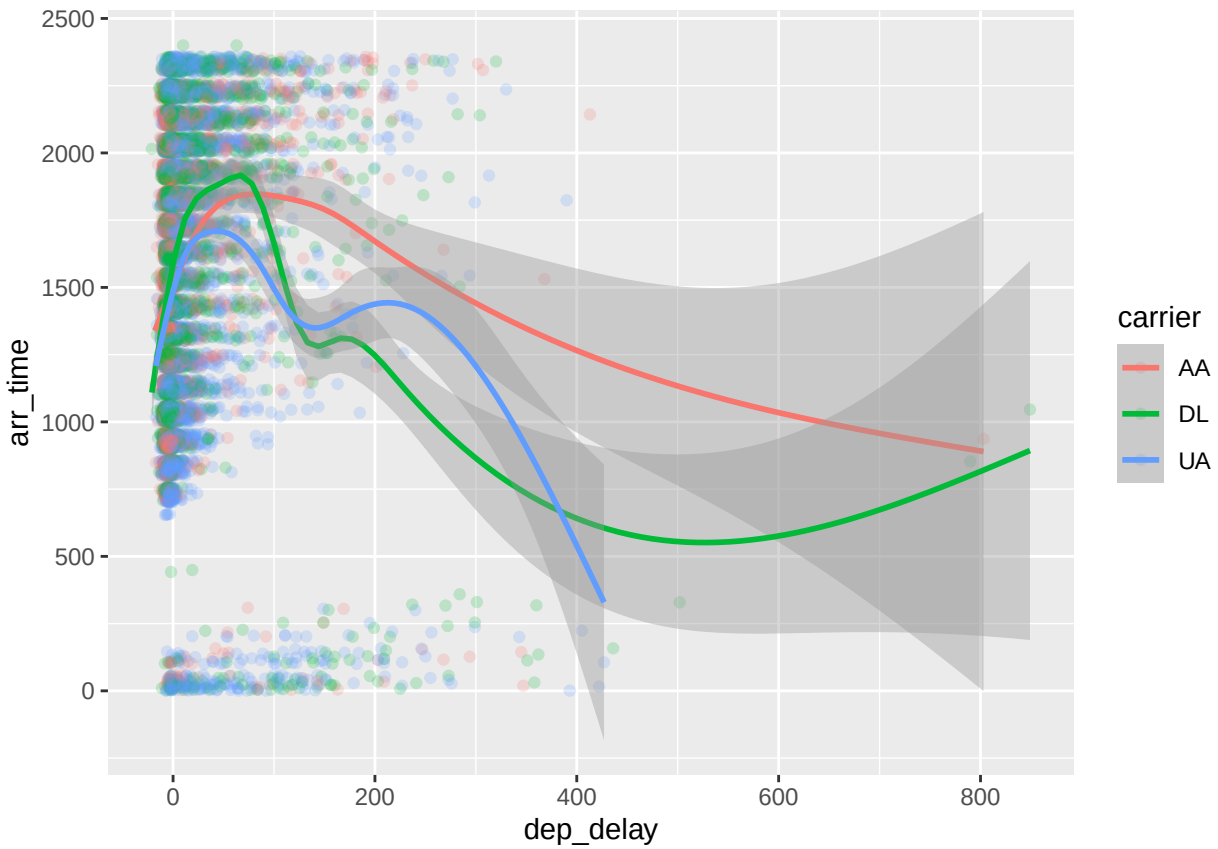


9. Replicate the following plot. **Hint:** The data frame plotted only contains flights from American Airlines, Delta Airlines, and United Airlines, and the points are `colored` by `carrier`. Once you replicate the plot, determine (roughly) what the cutoff point is for departure delays where you can still expect to get to your destination on time.



Insert your answer here

```
nycflights%>%
  filter( carrier %in% c('AA', 'DL', 'UA'))%>%
  ggplot(aes(x = dep_delay, y = arr_time, color = carrier)) +
  geom_point(alpha = 0.2) +
  geom_smooth()
```


2500
2000
1500
1000
500
0
arr_time
0
200
400
600
800
dep_delay
carrier
AA
DL
UA